
CLASSIFYING FAKE NEWS

TECHNICAL REPORT – JULY 12, 2026

GITHUB: [HTTPS://GITHUB.COM/KARLAUGUSTNYBO/FAKE-NEWS-PROJECT](https://github.com/karlaugustnybo/fake-news-project)

Daniel Jensen : nfh206
Department of Computer Science
Copenhagen University
nfh206@alumni.ku.dk

Tøger Holm Lyngbo : nvk539
Department of Computer Science
Copenhagen University
nvk539@alumni.ku.dk

Karl August Krogh : kcj878
Department of Computer Science
Copenhagen University
kcj878@alumni.ku.dk

Part 1: Data Processing

To begin our analysis, we preprocessed the FakeNewsCorpus (FNC) subset. However, due to the dataset's large size, we started by developing our data processing pipeline on a smaller sample.

Task 1 – Processing a smaller sample

Our data processing was mainly driven by the *pandas* library, as its many methods for common data-inspection tasks allows for clean and readable code. Using this, we first stored our data as a *DataFrame*, structuring it in a tabular form consisting of rows and columns using the `pd.read_csv()` function. We then proceeded by setting the index and filtering out bad columns like "Unnamed: 0". Afterwards, we used the `.isna()` method to inspect the fraction of missing values in each column, dropping those containing only NULLs.

For the remaining missing values, we used different imputation methods depending on the feature's datatype. For continuous features, we filled in using the median. For non-ordinal categorical features we added a "nan"-category indicating that a value was previously missing. For type and content, we dropped the rows with missing values, as we could not train on these.

With no missing values, we started cleaning the features containing language, but most specifically content and title. We used 3 steps to clean each text: First we used built-in python methods to split the string into words, strip the words of symbols on the sides, and convert the strings to lowercase. Then we used the built-in regular-expression library *re* to filter out symbols inside words which we didn't want, and then to convert certain words to special tokens like dates, numbers, e-mails, and URLs. After that, we used the *NLTK-Stopwords* package to filter out english stopwords, and finally, we used the *NLTK-Porter-Stemmer* to stem every word before aggregating them back into a single string. Lastly, we applied our cleaning function to the language features using the *pandas* `.apply()` method, concluding our preprocessing pipeline.

Furthermore, using these tools, we were able to count the size of our vocabulary before and after cleaning, which showed first a reduction from 30715 unique tokens down to 17072 after our regex cleaning (44.42% reduction), then 16915 after removing stopwords (0.92% reduction), and then just 11700 after stemming (another 30.83% reduction). This brings the total reduction rate to 61.91%.

Task 2 – Processing a larger subset

In order to apply our pipeline to the larger subset, we utilized the *polars* library that is build upon Rust and the Apache Arrow memory model and thereby operates in a more reasonable runtime than Pandas. We also used batches of 50,000 rows to avoid over-allocating RAM due to the subset's large size. Additionally, during this step we switched to *pystemmer*, as stemming took a lot of computation, and this was significantly faster than with the NLTK PorterStemmer.

Task 3 – Exploring the data

After preprocessing and converting back to a Pandas *DataFrame*, we searched the data for interesting patterns and limitations. We found several of these:

Zip's Law: We first investigated the word-frequencies by plotting a bar-chart of the 10,000 most common words in our corpus as seen in *Appendix A, fig. 3*. The cleaned results show a less steep asymptote in comparison to the uncleaned Zipf's plot (*Appendix A, fig. 4*.) We also decided to plot the top 100 words after cleaning to see if any patterns occurred (*fig. 1*.) Unsurprisingly, we found that <NUM> was by far the most common token, but more interestingly was that we observed clusters of words related to specific domains such as politics, geography and journalism, giving us insight into the kinds of articles the FNC represented.

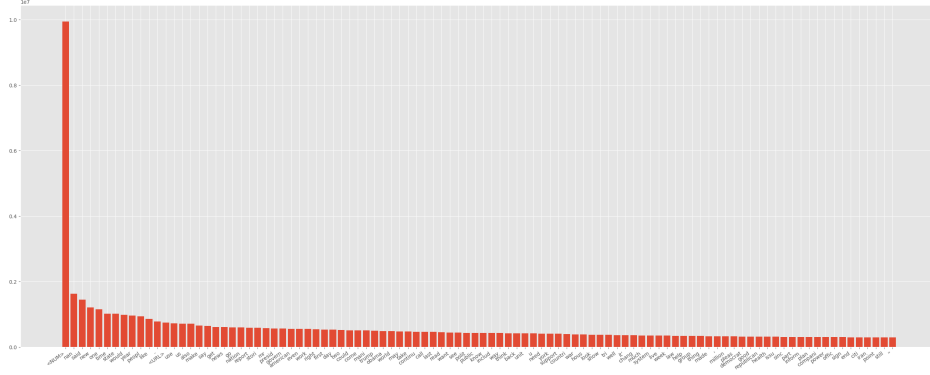


Figure 1: Zip's plot over 100 most common words in FNC after cleaning.

LIX-Scores: Lix is a widely used measure for the readability of a text. After computing and plotting the lix scores of the articles in our corpus, we suspected that they were log-normally distributed. Therefore, we plotted a histogram of log-transformed lix scores (see *appendix A, fig. 6*). The histogram confirmed that the log-transformed scores were normally distributed with a mean of ≈ 4.32 . This highlights another limitation of the data; for natural text seen in books, a log-lix number of 3.93-4.11 is viewed as abnormally large.¹ The large mean of our distribution indicates that the articles of our corpus are unrepresentative of natural, book-like text possibly limiting our results – even after preprocessing.

Duplicate Articles: One of the inherent problems we found with the dataset while working with it, is that it contains clusters of outliers that act as noise when making predictions. For example, in the original dataset, *about 7.6% of all rows were duplicates of the same 4 articles*. We found this pattern when investigating the LIX-coefficients of all raw articles. As seen in *Appendix A, fig. 6* by the clusters of values in uncommon places. We investigated the corresponding rows and found that they were duplicated values rather than generally common-occurring LIX-scores. Sadly, we discovered this problem after training our models, making us unable to counteracted it in preprocessing. In our discussion, we will recall them as their appearance should have consequences on the quality of our results.

Article lengths: Another investigation we did was looking at the average length (number of words) of the articles and titles. We found that the cleaned articles were quite long, consisting of ≈ 273 words on average (plus ≈ 6 words in title), which becomes a relevant limitation later when discussing the distribution of the LIAR dataset. We also plotted the distribution of article lengths, and surprisingly found that it appeared to exhibit an *exponential distribution*, with smaller articles being the most frequent group, but due to their small size, still being the minority (see *appendix A, fig. 7*). We even found articles with less than 3 words, demonstrating a limitation of our cleaning pipeline – namely that it may strip some articles of most their information, making for noisy data points. On the opposite pole, we found outlier-articles containing 20,000+ words after cleaning, which might also act as noise.

Task 4 – Splitting the dataset

To split the dataset, we used the standard scikit-learn `train_test_split` method, with the important parameter `stratify=labels` in order to make the train-val-test splits more representative of their shared, underlying distribution. Using a standard 80/10/10 split, we were now ready to train some classifiers.

Part 2: Simple Logistic Regression Model

As a baseline for our more complex models, we trained a simple logistic regression classifier on our cleaned FakeNews data, using a bag-of-words representation of the `content` column.

Task 1 – Grouping the labels

As we will only concern ourselves with binary classification, we needed to characterize each label of the corpus as "fake" or "real". Since many of the labels are of ambiguous reliability, we have decided to rule our data with an iron fist and label them all fake, as they might simply act as noise for the model and reduce the interpretability of its predictions. Furthermore, some labels such as "NaN" were pure noise not yet filtered out after the cleaning process. Thereby, we decided only to keep `reliable` as our real label, `political`, `bias`, `fake`, `unreliable`, `rumor`, `conspiracy`, `clickbait`, `junksci`, `hate`, and `satire` as fake labels, and the rest as ambiguous labels to be dropped. With that said, this selection is still limited as many classes such as `political`, `bias`, `clickbait`, or `satire` may consist partially of

¹LIX Readability Score. <https://readable.com/readability/lix-rix-readability-formulas/>

reliable and unreliable articles. This may send false signals to our models, complicating the learning process and leading to worse results. A thorough test was done of different label combinations, but adding political or bias as real, resulted in significantly degraded F1 performance. As showed later, this effects the models ability to correctly classify very political articles, though they do learn some distinction between fake- and real political articles.

Task 2 – Implementing the logistic regression classifier

Now, we begun training a Logistic Regression Classifier using L2-regularization, lambda-coefficient $C=1.0$, and iterations `max_iter=1000` on a bag-of-words representation of the content column. Fitting the vectorizer caused RAM problems, so to combat this we fitted on only 30% of the training data – approximately 216K articles. Throughout our analysis, we will only be including the 10K most frequent tokens in our BOW and 100K TF-IDF representations. In this context, a 30% sample is plenty: The unseen tokens of the 70% will likely be removed anyways, so the loss in performance is negligible. Similarly, when vectorizing the data, we reduced RAM consumption by processing the data in batches of 50K articles and performing garbage collection in the process. Lastly, to account for class imbalance, we included class weight coefficients in the loss function of the training process.

This model achieved an F1-score of 0.89 and 0.96 on the "real" and "fake" class respectively with fairly balanced precision and recall. This results in a macro F1 of 0.93 and an accuracy of 0.94. These results act as a good baseline, as the logistic classifier is a simplistic in that it assumes the fake news problem is linearly separable.

Task 3 – Including meta-data

As an experiment, we tried including meta-data columns in our classification problem as they may add previously unseen information to the data, and in turn help our model classify articles more accurately. We found that it would be fitting to include the categorical columns, the natural language columns, and the content column of the corpus as these can easily be encoded and might store useful patterns.

To encode the categorical columns `domain`, `authors`, and `source`, we limited them to the top 100 values and made one-hot encodings from these. Then, we aggregated the natural language columns `title`, `meta_keywords`, `meta_description`, and `tags`. This allowed us to construct a bag-of-words representation from the meta data and the content column separately. Hereafter, we were left with a DataFrame of 12,206 numerical columns. On this data, we trained a logistic model with the same parameters as before for comparability. This resulted in F1 scores of 0.92 and 0.97 on the real and fake classes, respectively, and a macro F1 and accuracy of 0.95 and 0.96. Thus, this extra information seems to substantially increase the performance, although further analysis is required to identify the features contributing to these predictions. For the rest of the project, we limit ourselves to just the main-text (content column) data as this will allow our models to be applicable for other data such as the LIAR dataset (see part 4).

Part 3: Advanced Models

To maximize performance, we implemented a long range of models on the entire dataset. We mainly focused on two different ML algorithms: A *Support-Vector-Machine Classifier* using TF-IDF features and a finetuned state-of-the-art auto-encoder (*DistilBERT-Base-Uncased*). Moreover, we also implemented a logistic regression using TF-IDF features, a Multinomial Naive-Bayes classifier, an XGBoost classifier, and a Support-Vector-Machine Classifier using Singular-Value-Decomposition. We will now report the central details of our two main models.

SVM Classifier Using TF-IDF

For the SVM we used the SciKit-Learn `TfidfVectorizer` with the chosen parameters:

- `max_features = 100_000`
- `ngram_range = (1,2)`: For both max features and ngram range we did some quick tests on 10K articles that showed diminishing returns for ngram range (1, 3) and max features over 80K. Thus, we decided on the (1, 2) range to decrease training time and the 100K feature cap to account for the vastness of the 1 million articles.
- `sublinear_tf = True`: We use log scaling on term frequencies to reduce the impact of very frequent words. Words that appear 10 times still matter more than words that appear 3 times, but the difference is compressed.
- `min_df = 3` and `max_df = 0.95`: We remove both words and ngrams that are present in less than 3 articles or over 95% of the articles.

As with the logistic regression, we only fitted on a 30% subset of the training data to decrease the memory consumption of the dataset. We also used 50K article chunks when transforming the three splits, as well as garbage collection to reduce memory load.

We chose to use `LinearSVC` as our model architecture because it is faster than `sk-learns` standard `SVC` for larger datasets and works well for our high-dimensional TF-IDF features. Before training the final model, we performed a logarithmic

sweep through C values and found $C \approx 0.372$ to maximize F1 score on the validation set. Lastly we weighted the loss function under training, to compensate for the 3:1 class imbalance.

To test for overfitting we did a qualitative test with handpicked articles from the internet. We tested the model on 3 articles from Reuters (one neutral, one mildly political, and one highly political), 1 from The New Yorker (biographic article), 1 from The Onion (satire), and 1 from InfoWars (2020 election fake news). Table 1 below shows the predictions and the SVM Score, which represents the distance between the article’s placement in the hyperspace, and the decision boundary. We can see that the model generalized well to these real world examples, with the only mistake being the highly political article being classified as fake although it is factually true. This makes sense as we have grouped the political label in the fake category but even then, the model manages to correctly predict the semi-political article as real. Moreover, we can see from the SVM score that the model is uncertain of these predictions, as the distances

to the decision boundary are short. Conversely, the model clearly and confidently categorizes the fake article as fake. However, it should be noted that the satire article is predicted as fake, which corresponds to our chosen labels but arguably shouldn’t be the case at all times. Further analysis on what the SVM predicted sorted by each label can be found in Appendix A, table 3.

Category	Source	Prediction	SVM Score
Normal	Reuters	REAL	-0.4282
Semi-political	Reuters	REAL	-0.1653
Very-political	Reuters	FAKE	0.0825
Biographic	The New Yorker	REAL	-0.1409
Satire	The Onion	FAKE	0.9307
Fake-news and political	InfoWars	FAKE	1.2706

Table 1: SVM Classification Results Across Handpicked Articles

DistilBERT Fine-Tuning & Neural Network Classifier

We also wanted to try using a transformer-based model as it has been redefining the state-of-the-art for the last few years. To do this, we decided to fine-tune a pretrained transformer from the *HuggingFace Transformers Platform*.

Our fine-tuning works by adapting a complex auto-encoder that has been previously trained on a large corpus, until it has gained a complex, bi-directional understanding of how language is used. This model, however, has only been trained to fill in missing words from a corrupted document using semi-supervised learning in a process known as masked-language-modeling (MLM), and thus we need to retrain its parameters to match the needs of our problem, i.e., classifying an article as fake or not. Rather than training our own encoder from scratch, we take the pretrained model, with its current contextual understanding, and polarize its parameters to fit our problem using supervised labels, a process known as *transfer-learning*.

In total, our pipeline for the *DistilBERT-base-uncased* classifier was like so: We chose a tokenizer with padding, truncation, specials, and a max size of 512 tokens (the max token limit of DistilBERT). For our auto-encoder, we used max-mean-pooling, a classification head consisting of 4 linear, ReLU-activated layers ($1024 \rightarrow 256 \rightarrow 16 \rightarrow 1$), and 4 dropout layers ($0.5 \rightarrow 0.35 \rightarrow 0.2 \rightarrow 0.2$) to classify each article. This was chosen because we wanted to give our model the opportunity to capture rich patterns in the data, but with high dropout to stop it from overfitting. We used Erda’s GPUs to train the model for 3 epochs on the *uncleaned data*², with a learning rate of $2 \cdot 10^{-5}$, a weight-decay of 0.01, *binary-cross-entropy with logits* loss, the AdamW optimizer, and a *randomized data-loader* with batch-size 64. The rest of the hyperparameters, we simply chose because they are commonly-used industry standards.

After training, we evaluated the model on the validation set, reaching an F1-score of 97.44%, a macro F1-score of 94%, and an accuracy and weighted F1-score of 95%, with slightly higher precision on the real-class and slightly higher recall on the fake class.

We later tried training a DistilBERT-Base-Uncased classifier using the *HuggingFace Trainer API*, which proved to score higher than fine-tuning on our own using *PyTorch*. The results of this model are included in table 2. Specifically, the classifier scored a macro F1-score of 98.7%, as well as an accuracy and weighted F1-score of 99.1%. Due to time restraints a full analysis of the model wasn’t possible.

The complex models are both strong and limited. They have *high potential*, as their deep architecture allows for almost *universal-approximation*, meaning they can theoretically model the *oracle-distribution* very closely compared to more shallow models. With that said, SVMs and transformers are also limited: they take much longer to train, are less explainable due to their depth, and are more likely to overfit if not trained carefully. With a generous portfolio of models, we were now ready to test their performance on the FNC test set, as well as see how they generalize to foreign data.

²This was done to ensure the encoder has access to as much information as possible while matching the distribution it was trained on more accurately, as the pretrained tokenizer expects raw text. Consider the *Data and compute power* section on page 3 in the original paper: <https://arxiv.org/pdf/1910.01108>

Model	Test macro	Test acc.	NDM	NDA	LIAR macro	LIAR acc.
API DistilBERT	0.99	0.99	–	–	0.45	0.63
DistilBERT	0.94	0.96	–	–	0.53	0.65
LinearSVC TF-IDF	0.96	0.97	0.97	0.98	0.51	0.74
Logistic Regression BOW	0.93	0.95	0.92	0.93	0.41	0.64
Logistic Regression Meta	0.95	0.96	–	–	–	–
Logistic Regression TF-IDF	0.93	0.95	0.91	0.93	0.42	0.64
Multinomial N.B. BOW	0.85	0.90	0.86	0.88	0.52	0.60
XGBoost TF-IDF	0.92	0.94	0.91	0.93	0.39	0.65
SVD SVM TF-IDF	0.91	0.93	0.90	0.92	0.43	0.64

Table 2: Performance comparison on the FakeNews testset and the LIAR dataset

Part 4: Evaluation

To evaluate the performance of our models, we both used the test set generated from the FakeNewsCorpus and the whole LIAR dataset. Thereby, we could explore how our models generalize to unseen data. In *table 2*, we’ve summarized our results on both datasets.

Task 1 – Evaluation on the FakeNewsCorpus test set

When evaluating our models on the FNC test set, they all generalized very well. This might be a consequence of the similarity between the test set and the training set: It is unlikely that this test set exhibits sample bias in comparison with the training set as they are both generated from the same distribution.

Task 2 – Evaluation on the LIAR dataset

To test the models’ cross-domain performance, we tested them on a dataset fundamentally different than the FNC: The LIAR dataset. As we would only be testing on the LIAR dataset, we chose to use all the data for this purpose by concatenating the train, validation, and test set together. We chose to label *mostly-true* and *true* as real and all other labels as fake. When evaluating our models, they all struggled to generalize. Even more interestingly, most seemed only to predict the fake class. This will be discussed further in part 5.

Task 3 – Comparing performance between datasets

If we compare the performance of our main models, we get the following:

The logistic regressor generalized poorly to the LIAR dataset, being biased towards the fake class. It obtained an F1-score of 78%, a macro-F1-score of 41%, an accuracy of 64%, and a weighted-F1-score of 52%.

The SVM didn’t generalize too well either. It got an F1-score of 0.85 on the fake class but only 0.17 on the real class, resulting in a macro F1 of 0.51 and an accuracy of 0.74. Especially the recall on the real class was low: Only 0.11. As with the logistic regression, this indicates a bias towards the fake class.

We also tried directly applying our DistilBERT auto-encoder on the LIAR dataset, and similar to the svm, it didn’t generalize too well, only obtaining an F1-score of 0.78 and a macro F1-score of 0.53, again indicating a clear imbalance in its predictions. Confirming this, we see in *Appendix A fig. 5 (b)* that the classifier guesses fake almost exclusively.

Data leakage: After evaluation, we found a likely source of data leakage: As described in part 1, the FNC included at least 7.6% duplicates, many of which we believe were not removed in preprocessing. After splitting our data randomly, the validation and test set thus likely contains duplicate articles from the train set. To account for this, we removed all articles in the test set also included in the train set, reducing the number of articles by 22%. We then reevaluated as many models as we had time for on this data. The resulting scores are seen in *table 2*, with *NDM*, *NDA* corresponding to *no-duplicate* macro F1 and accuracy respectively. Overall, we only see a small loss in performance after removing the duplicates, and surprisingly, the LinearSVC performed even better than before. We ran into trouble when trying to run DistilBERT on the non-duplicate test-set, as we could not import it onto our GPU-server in time.

Part 5: Conclusions

The evaluation shows a significant gap in generalization: While models achieve high performance on the FNC, they fail to generalize to the LIAR dataset, mainly predicting the fake class. We are confident that this discrepancy is not caused by model architecture, but by fundamental sample biases between the two domains:

Size difference: There is a great difference in article lengths between the two datasets. The LIAR dataset has a median content length of 99 characters whereas the FNC has a median length of 1814. If the models associate short

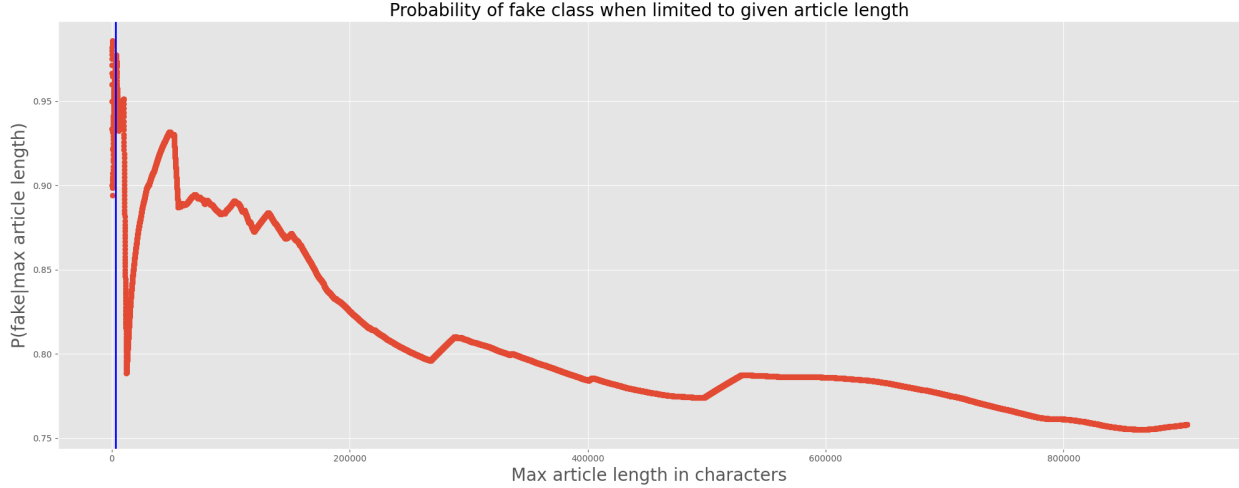


Figure 2: Probability of fake class for the subpopulation of the FakeNewsCorpus with the given maximum article length. The blue bar is the max article length of the LIAR dataset.

article lengths with the fake class, this biases their predictions in this direction. To test this hypothesis, we computed probabilities of an article in the FNC being "fake" given that its length is below certain thresholds. In *fig. 2*, this computation is performed for a range of values. A clear pattern appears: The probability of the fake class seems to decrease as articles of larger length are included in the computation. The blue line indicates the maximum article length of the LIAR dataset. It is clear that articles in the FNC shorter than this have a high chance of being "fake". This can be underlined by looking at specific values in the plot. We can limit the FNC article lengths to the quantile values of the LIAR dataset article lengths and compute the probabilities of the fake class in each resulting grouping. This gives us the probabilities 0.799, 0.879, and 0.931 for the first quantile, median, and third quantile respectively. The magnitude of these values substantiates our hypothesis: The change in article length might in fact cause a concept shift between the two datasets. In FakeNewsCorpus, the feature of short article lengths implies a high probability of the fake class but in LIAR, this is not necessarily the case. This is likely the greatest cause of the discrepancy between our results.

Prior probability differences: There might be a prior probability shift between the two datasets that could explain our models' performances. To test this hypothesis, we computed the fraction of fake labels in the datasets. These appeared to be 75.8% for the FNC and 64.8% for the LIAR dataset. This decrease in the proportion of fake labels may bias the evaluation, disproportionately reducing F1 scores due to diminished instances of the class.

Differences in dataset labels: There is a discrepancy between the labels of the two datasets. The LIAR dataset categorizes the articles based on *factual truth value*, on statements communicated to *appear truthful*, while the FakeNewsCorpus categorizes articles based on the "genre" of the articles. A "clickbait" article might be factually true although categorized as "fake". Moreover, the labels of the FakeNewsCorpus are purely generated from the domain of the articles.³ This causes our models to predict domains rather than factual truth, explaining the lack of generalization.

Time difference: The FNC was last updated in 2018. When predicting on older or newer data, this could result in a concept shift as the truth value of a specific statement can change over time. The LIAR dataset was last updated in 2017, and even this gap may have caused a concept shift.

At first, we found our models' poor performances quite surprising. One could expect that a dataset of a million articles should contain the richness needed to generalize to a foreign dataset. However, it appears that the sample bias in the data limits our models. Even the best model on the best dataset may fail under a concept shift. Therefore, we are confident that the main bottleneck of our classification performance is *the data*. Besides the above, this hypothesis is based on the fact that we trained a long list of models, all disproportionately predicting the fake class when tested on the LIAR dataset. To improve our predictions, we would therefore prioritize accessing better data. As described earlier, our results were likely limited by the fact that the FNC included problems such as large clusters of duplicates, unnatural language, and extremely short or long articles. This could more or less have been accounted for when preprocessing the raw data. Similarly, the data could have been improved by labeling from factual ground-truth rather than source-based alone, a consideration that could be addressed in future work.

³Maciej Szpakowski: Fake News Corpus, 2018. <https://github.com/several27/FakeNewsCorpus>

Appendix A – Figures

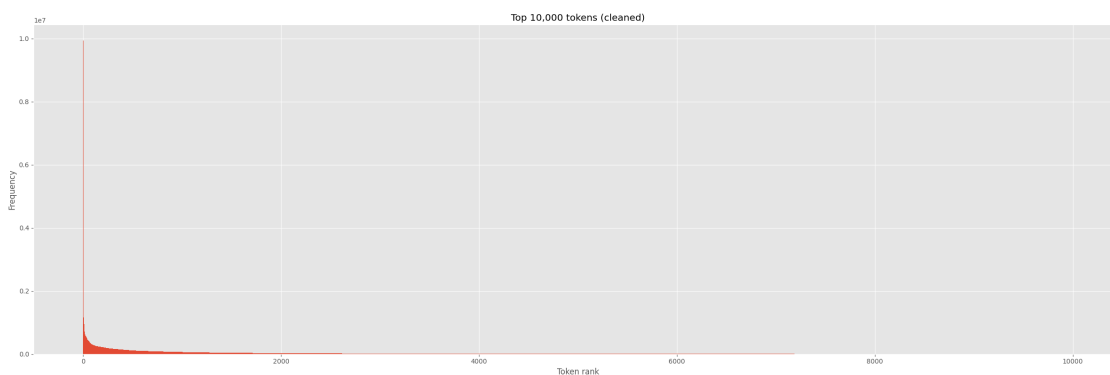


Figure 3: Zipf's plot over 10,000 most common words after cleaning.

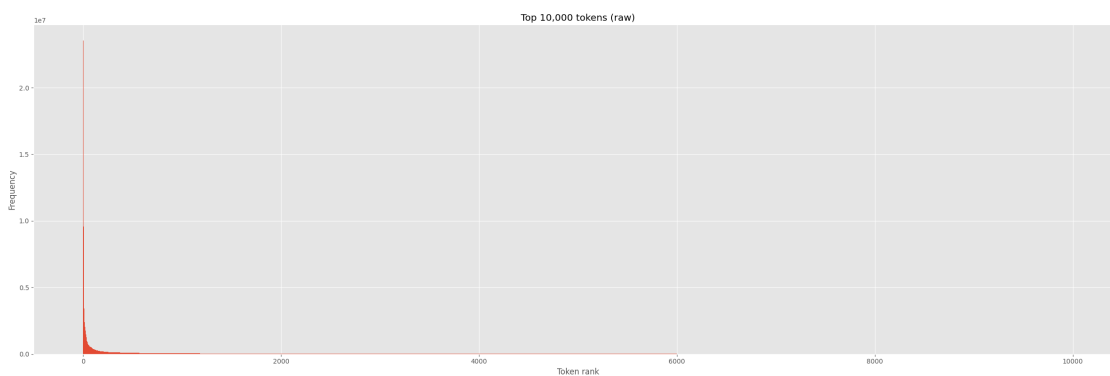
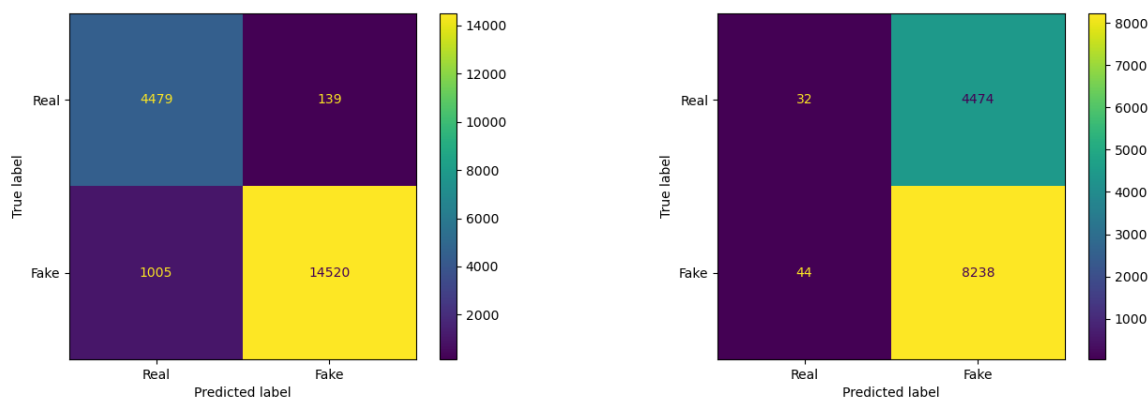


Figure 4: Zipf's plot over 10,000 most common words before cleaning.



(a) Performance on similar-sized FakeNews-Testset

(b) Performance on LIAR Dataset

Figure 5: Confusion Matrices For DistilBERT-Base-Uncased Classifier

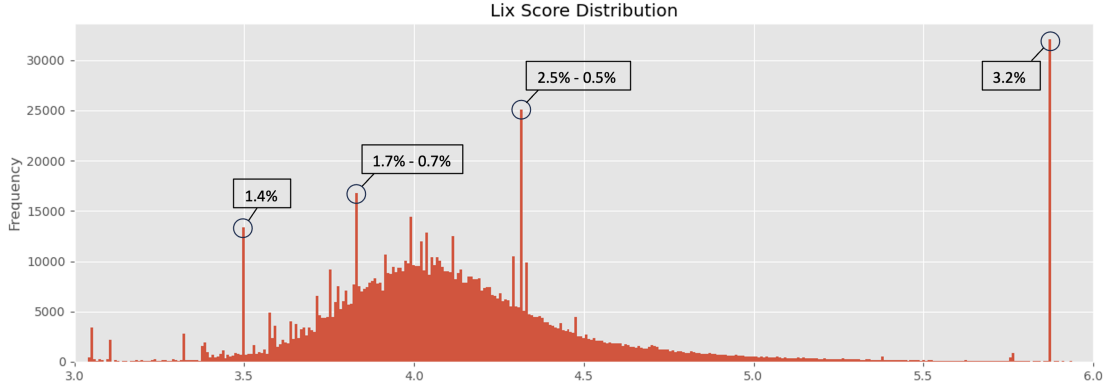


Figure 6: Distribution of Log-LIX-Scores

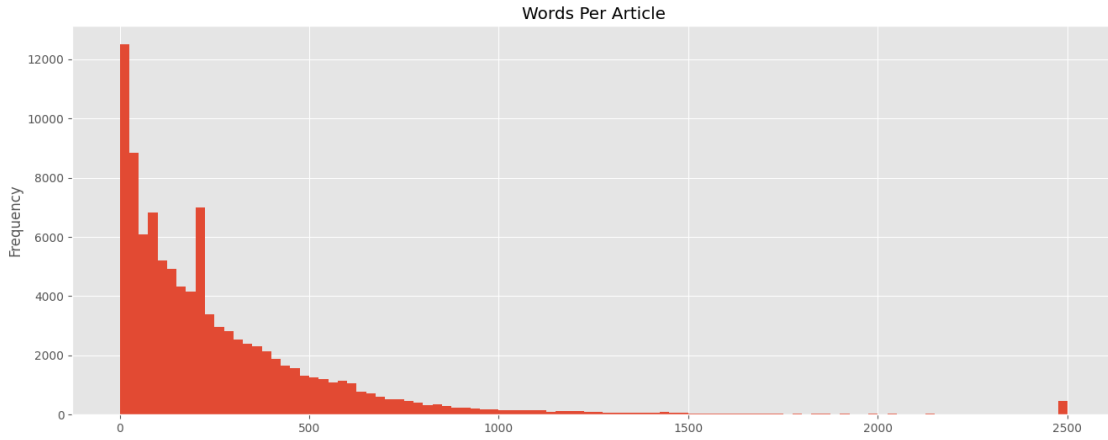


Figure 7: Distribution of Article Lengths in words, capped at 2500 words for better view.

type	count	mean guess	mean score
reliable	21856	0.054	-2.067
political	19410	0.965	1.207
bias	13267	0.985	1.573
fake	10491	0.982	1.941
conspiracy	9682	0.988	1.444
rumor	5708	0.970	1.289
unreliable	3645	0.990	1.196
clickbait	2668	0.955	1.184
satire	1394	0.958	1.088
junksci	1388	0.992	1.332
hate	859	0.981	1.476

Table 3: SVM predictions grouped by label.

width=!,height=!,pages=1